

4. CPU Scheduling or Process Scheduling

Dr. Arun Kumar

Dept. of CSE, NIT Rourkela

CPU Scheduling

▶ CPU Scheduling

- ▶ Preemptive Scheduling
- ▶ Non-Preemptive Scheduling

▶ Non-Preemptive Scheduling

- ▶ When a process completes its execution.
- ▶ When a *process leaves CPU Voluntarily* for I/o operation or event.

CPU Scheduling

▶ Preemptive Scheduling

- ▶ High **priority** process.

- ▶ Process *switches due to **time quantum*** expiration.

Scheduling Terminology

- ▶ **Bust Time/Execution Time/Running Time:** it is the time a process requires for *running on CPU*.
(User has no say in it)
- ▶ **Waiting time:** amount of time a process has been *waiting in the ready queue* – want to *minimize*.
- ▶ **Arrival Time:** When a process *enters the Ready State*.
- ▶ **Exit Time/Terminating Time:** Time when a process *completes execution and exist from the system*.

Scheduling Terminology

► **Turnaround time:** amount of time to execute a particular process – want to *minimize*.

► $T.A.T = E.T - A.T = B.T + W.T$

► **Response time:** amount of time it takes from when a job was *submitted* until it initiates its *first response* (output) – want to *minimize*.

Criteria for CPU Scheduling Algo:

- ▶ **Average Waiting Time:** – want to *minimize*.
 - ▶ **Average Response Time:** – want to *minimize*.
 - ▶ **CPU utilization:** i.e. CPU usage – want to *maximize*.
 - ▶ **Throughput:** number of processes that complete their execution per time unit – want to *maximize*.
- 

First-Come, First-Served (FCFS) Scheduling

- ▶ Simplest scheduling algorithm.
- ▶ Assigns the process which arrives first.
- ▶ Easy to understand and can be easily implemented using Queue data structure (FIFO).
- ▶ Always *non-preemptive* in nature.
 - ▶ *Can't be interrupted*

First-Come, First-Served Scheduling

- Schedule: order of arrival of process in ready queue

Example:

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

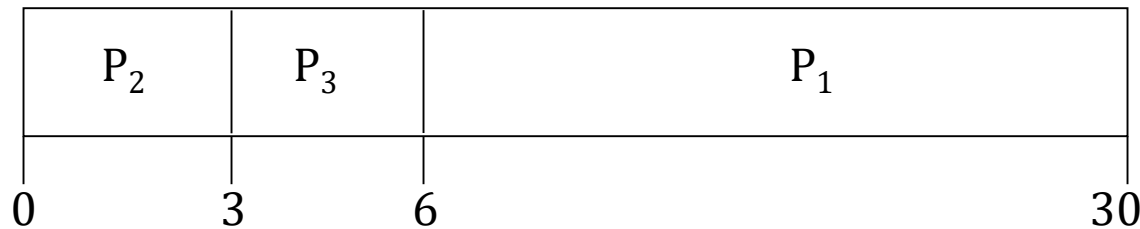
- Suppose that the processes arrive in the order: P_1, P_2, P_3 . The Gantt Chart for the schedule is:



- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$

FCFS Scheduling

- ▶ Suppose that the processes arrive in the order P_2, P_3, P_1 .
- ▶ The Gantt chart for the schedule is:



- ▶ Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- ▶ Average waiting time: $(6 + 0 + 3)/3 = 3$

FCFS Scheduling

▶ **Convoy effect:** short I/O intensive processes behind long CPU intensive process.

▶ *Smaller process has to wait for longer time for bigger process to release CPU.*

▶ Example- Minister Convoy, Petrol Pump – Truck versus Bike

▶ Advantage

▶ Simple to understand

▶ Easy to use.

▶ Simple to implement by FIFO queue.

▶ Google drives sync, downloads, background processes, App updates

FCFS Scheduling

► Disadvantage

- Suffers from *convoy effects*.
- Normally higher average waiting time. R_T
- No consideration to *priority* or burst time.
- It is not a good choice for interactive systems.

► Does FCFS suffers from starvation?

- No – CPU is not biased.

Shortest-Job-First (SJF) Scheduling

- ▶ Schedule: Order of the CPU burst of the processes (Shortest Job first).
- ▶ Two schemes:
 - ▶ Non-Preemptive *SJF*/Shortest Job Next (*SJN*)
 - ▶ Once CPU given to the process it cannot be preempted until completes its CPU burst.
 - ▶ Preemptive *SJF*/Shortest Remaining Time First (*SRTF*)
 - ▶ If a new process arrives with CPU burst length less than remaining time of current executing process, preempt.

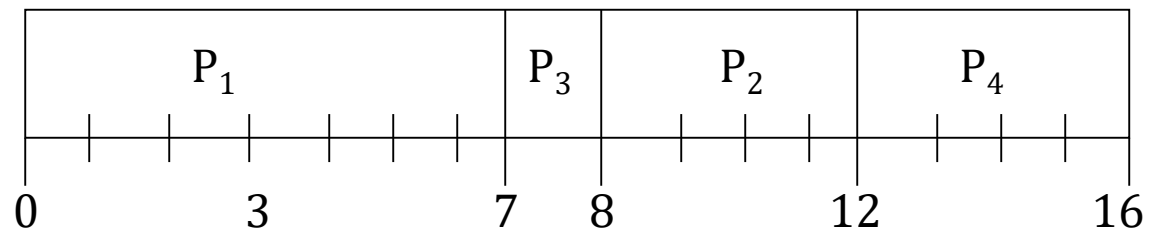
Shortest-Job-First (SJF) Scheduling

- ▶ CPU is assigned to the process that has smallest Burst Time requirement (***no priority, no seniority***)
- ▶ In case of tie, *FCFS* is used to break the tie.
- ▶ It can be used for *non-preemptive* and *preemptive* approach.
- ▶ Preemptive version (***SRTF***) is also called as ***optimal*** as it guarantees minimal average waiting time (***Greedy approach***).

Non-Preemptive SJF (SJN)

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>	<u>T.A.T</u>	<u>W.T</u>
P_1	0	7	7	0
P_2	2	4	10	6
P_3	4	1	4	3
P_4	5	4	11	7

► Gantt Chart:

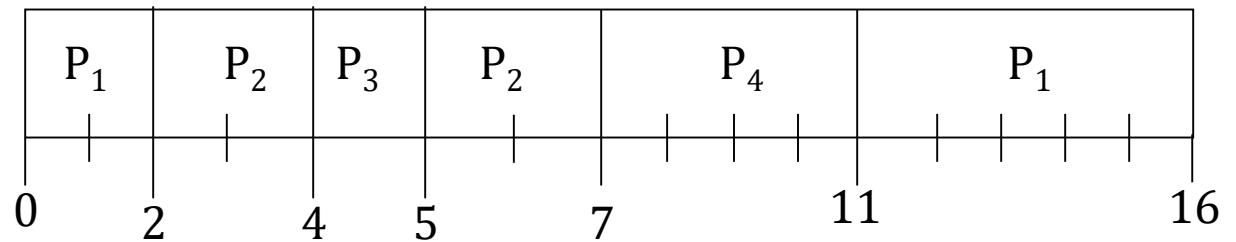


► Average waiting time = $(0 + 6 + 3 + 7)/4 = 4$

Preemptive SJF (SRTF)

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>	<u>T.A.T</u>	<u>W.T</u>
P_1	0	7	16	9
P_2	2	4	5	1
P_3	4	1	1	0
P_4	5	4	6	2

► Gantt Chart:



► Average waiting time = $(9 + 1 + 0 + 2)/4 = 3$

Example

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
<i>P1</i>	0	16
<i>P2</i>	2	8
<i>P3</i>	4	12
<i>P4</i>	6	2
<i>P5</i>	8	4

- ▶ Draw Gantt chart and find the average waiting time in each of the following cases:
 - ▶ SRTF Scheduling
 - ▶ SJN Scheduling
 - ▶ FCFS Scheduling

Answer to Example

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>	<u>T.A.T</u>	<u>W.T</u>
<i>P1</i>	0	16	42	26
<i>P2</i>	2	8	10	2
<i>P3</i>	4	12	24	12
<i>P4</i>	6	2	2	0
<i>P5</i>	8	4	8	4

► SRTF



► Average waiting time = $(26+2+12+0+4)/5$
 $= 8.8$

Answer to Example

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>	<u>T.A.T</u>	<u>W.T</u>
<i>P1</i>	0	16	16	0
<i>P2</i>	2	8	28	20
<i>P3</i>	4	12	38	26
<i>P4</i>	6	2	12	10
<i>P5</i>	8	4	14	10

► SJN



► Average waiting time = $(0+20+26+10+10)/5$
 $= 13.2$

Answer to Example

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>	<u>T.A.T</u>	<u>W.T</u>
<i>P1</i>	0	16	16	0
<i>P2</i>	2	8	22	14
<i>P3</i>	4	12	32	20
<i>P4</i>	6	2	32	30
<i>P5</i>	8	4	34	30

► FCFS



► Average waiting time = $(0+14+20+30+30)/5$
 $= 18.8$

Answer to Example

Scheduling Algorithm	Average Waiting Time
FCFS	18.8
SJN	13.2
SRTF	8.8

- ***No control over running time (Burst Time) – 42 units***
- ***Way to allocate to CPU has changed – changed the average waiting time***

Shortest Job First

Advantage:

- ▶ ***SRTF is Optimal***: Minimum Average waiting time. (Pure greedy approach)
- ▶ ***A standard*** for other algorithms in terms of average waiting time.
- ▶ Better ***average response time*** compared to FCFS.

Shortest Job First

Disadvantage:

- ▶ This algorithm can not be implemented - Difficult to know the CPU burst of processes before their execution.
 - ▶ ATM Transaction
- ▶ Process with longer CPU burst time may be starved.
- ▶ No priority, process with large burst time will have poor response time.

Priority Scheduling

- ▶ ***FCFS*** is effectively priority scheduling where priority is defined on ***arrival time***.
- ▶ ***SJF*** is effectively priority scheduling where priority is defined on ***next CPU burst time***.
- ▶ A ***priority number*** (integer) is associated with each process.
- ▶ The CPU is allocated to the process with the ***highest priority***

Priority Scheduling

- ▶ Some schemes use *smallest integer $\equiv$ highest priority*.
- ▶ In case of tie, *FCFS* is used to break the tie.
- ▶ *No effect* of Arrival Time or Burst Time.
- ▶ Supports both *non-preemptive* and *preemptive* versions.

Priority Scheduling

Advantage:

- ▶ Sense of priority specially for *system processes*.
- ▶ *Runs important processes first* even if it is a *user process*.

Priority Scheduling

Disadvantage:

- ▶ Problem: **Starvation** – low priority processes may never execute (higher priority processes may keep coming).
- ▶ Solution: **Aging** – as time progresses increase the priority of the process.
 - ▶ e.g. in every 10ms increase the priority of process by 1

Round Robin (RR) Scheduling

- ▶ This algorithm is designed especially *time-sharing* systems.
- ▶ It is similar to *FCFS*, but *preemption* is added to enable the system to switch between the processes.
- ▶ Each process gets a *time quantum* or *time slice*, usually *10-100 milliseconds*.
- ▶ After this *time has elapsed*, the process either *terminates* or *preempted* and *added to the end of the ready queue*.
- ▶ Ready queue is treated as *circular queue*.

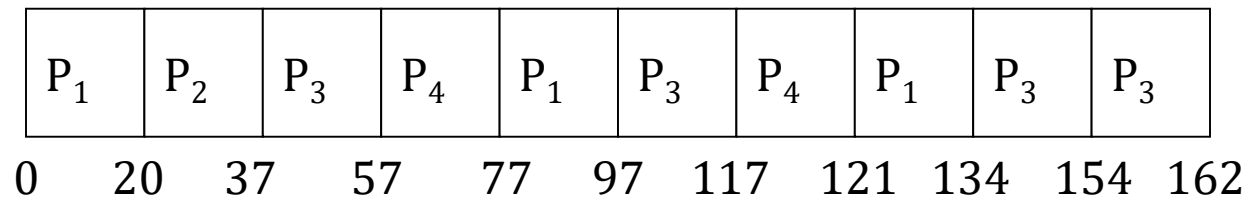
Round Robin (RR) Scheduling

- ▶ To implement RR, ready queue is treated as *FIFO* where new processes are added to the ***tail of the ready queue***.
- ▶ If there are n processes in the ready queue and the *time quantum* is q , then each process gets $1/n$ of the CPU time in chunks of atmost q time units.
- ▶ No process waits more than $(n-1)q$ time units.
 - ▶ Example: **5 processes** and time quantum of **20ms**, each process will get up to **20ms every 100ms**.

Example: RR with $q=20$

<u>Process</u>	<u>Burst Time</u>
P_1	53
P_2	17
P_3	68
P_4	24

► The Gantt chart is:



Round Robin (RR) Scheduling

Advantage:

- ▶ Typically, *higher average turnaround* than *SJF*, but better *response*.
- ▶ It is used for *time-sharing systems*, *client server* architecture and *interactive systems*.
- ▶ Similar to *SJF* implementation.
- ▶ *FCFS + Preemption = RR*

Round Robin (RR) Scheduling

Disadvantage:

- ▶ *Longer process may starve.*
- ▶ *Performance depends heavily on the Time Quantum.*
 - ▶ *q large $\Rightarrow$ behaves similar to **FCFS***
 - ▶ *q small $\Rightarrow$ more **context switch overhead**.*
- ▶ *No idea of priority.*

Modified RR Scheduling

- ▶ Based on *variable time* quantum.
- ▶ A *preempted process*, on its next turn will use *an increased time quantum*.
- ▶ RR Scheduling
 - ▶ P_1 : 300ms, $q=20$ ms, time per context switch=2ms
 - ▶ *Number of preemptions* due to this process = **14**
 - ▶ *Preemption overhead* = **28ms**

Modified RR Scheduling

► Modified RR Scheduling – *q is doubled* in next turn.

► P_1 : 300ms, $q=20\text{ms}$ (initial), time per context switch=2ms

► *Number of preemptions* due to this process = 3.
($20+40+80+160=300$)

► *Preemption overhead* = 6ms

Highest Response Ratio Next (HRRN) Scheduling

The algorithms we studied have problems:

- ▶ **FCFS (First Come First Serve):** Long processes make short ones wait (Convoy effect).
- ▶ **SJF (Shortest Job First):** Favors short jobs but long jobs may starve (never get CPU).
- ▶ **HRRN balances both.**

Highest Response Ratio Next (HRRN) Scheduling

- ▶ It uses a formula called **Response Ratio**:
 - ▶ Response Ratio (RR) =
$$\frac{\text{Waiting Time} + \text{Burst Time}}{\text{Burst Time}}$$
 - ▶ **Waiting Time (WT)**: How long process has been waiting.
 - ▶ **Burst Time (BT)**: How much time it needs to complete.
- Rule:** Pick the process with the highest Response Ratio next.

Highest Response Ratio Next (HRRN) Scheduling

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
<i>P1</i>	0	3
<i>P2</i>	2	6
<i>P3</i>	4	4
<i>P4</i>	6	5
<i>P5</i>	8	2



Highest Response Ratio Next (HRRN) Scheduling

Advantages:

- ▶ **No starvation** → Long processes will eventually get very high RR and get selected.
- ▶ **Fair** → Balances between short and long processes.

Multi level Queue Scheduling

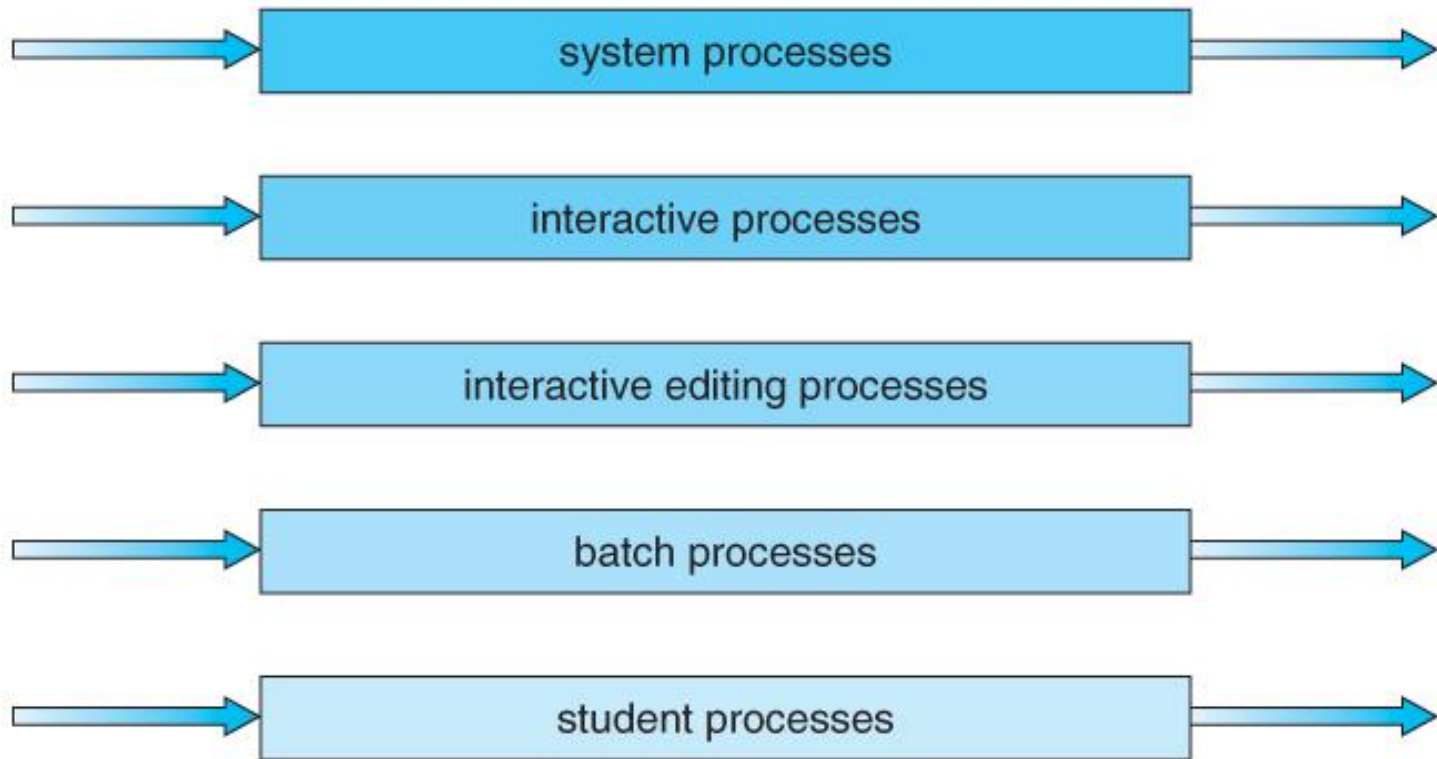
- ▶ Processes are classified into different groups.
 - ▶ **Foreground Processes** (Interactive processes)
 - ▶ **Background Processes** (Batch processes)
- ▶ *Ready queue* is partitioned into **separate queues**.
(In reality, we have more than 1 queues)
- ▶ *Each queue* holds a particular *category of processes*.
- ▶ *Each queue* can implement *whichever scheduling algorithm* is **most appropriate for that type of processes**.

Multi level Queue Scheduling

- ▶ *Foreground queue* might be scheduled by an *RR algorithm*.
- ▶ *Background queue* might be scheduled by an *FCFS algorithm*.
- ▶ Another possibility is to time slice among queues.
 - ▶ *Foreground queue* may be given 80 % CPU time and *Background queue* may get 20% of CPU time.

Multi level Queue Scheduling

highest priority



lowest priority

Multi level Queue Scheduling

Multilevel Queue Scheduling

- ▶ *Processes* are *permanently assigned to one queue*, based on process properties like *memory size, process priority* or *process type*.
- ▶ *Scheduling* must be done ***between the queues***.
- ▶ *Priority scheduling*; But possibility of *starvation*.
- ▶ *Time slice* – each queue gets a certain amount of CPU time which it can *schedule amongst its processes*.

Multilevel Feedback Queue Scheduling

- ▶ In *multilevel queue scheduling* processes were *permanently assigned to separate queues* (They can't move b/w queues.)
 - ▶ This has *low scheduling overhead* but it is *inflexible*.
- ▶ The idea with *multilevel feedback queue scheduling* is to *separate processes* according to the *characteristic of their CPU bursts* and they should be able to *jump b/w queues*.
- ▶ For Example: if a process uses *too much CPU time*, it will be *moved back to a lower-priority queue*.

Multilevel Feedback Queue Scheduling

- ▶ ***Shortest Job First (SJF)*** can optimize *turnaround time*, however, it is not possible to know the *CPU burst time of the processes prior to their execution*.
- ▶ ***Round Robin (RR)*** reduce *response time* but are *terrible for turnaround time*.

Multilevel Feedback Queue Scheduling

► Objectives:

- *To optimize turnaround time.*
- *To minimize response time (to make a system feel responsive to interactive users).*

THE CRUX:

HOW TO SCHEDULE WITHOUT PERFECT KNOWLEDGE?

How can we design a scheduler that both *minimizes response time* for interactive jobs while also *minimizing turnaround time* **without a priori knowledge of job length?**

MLFQ: Basic Rules

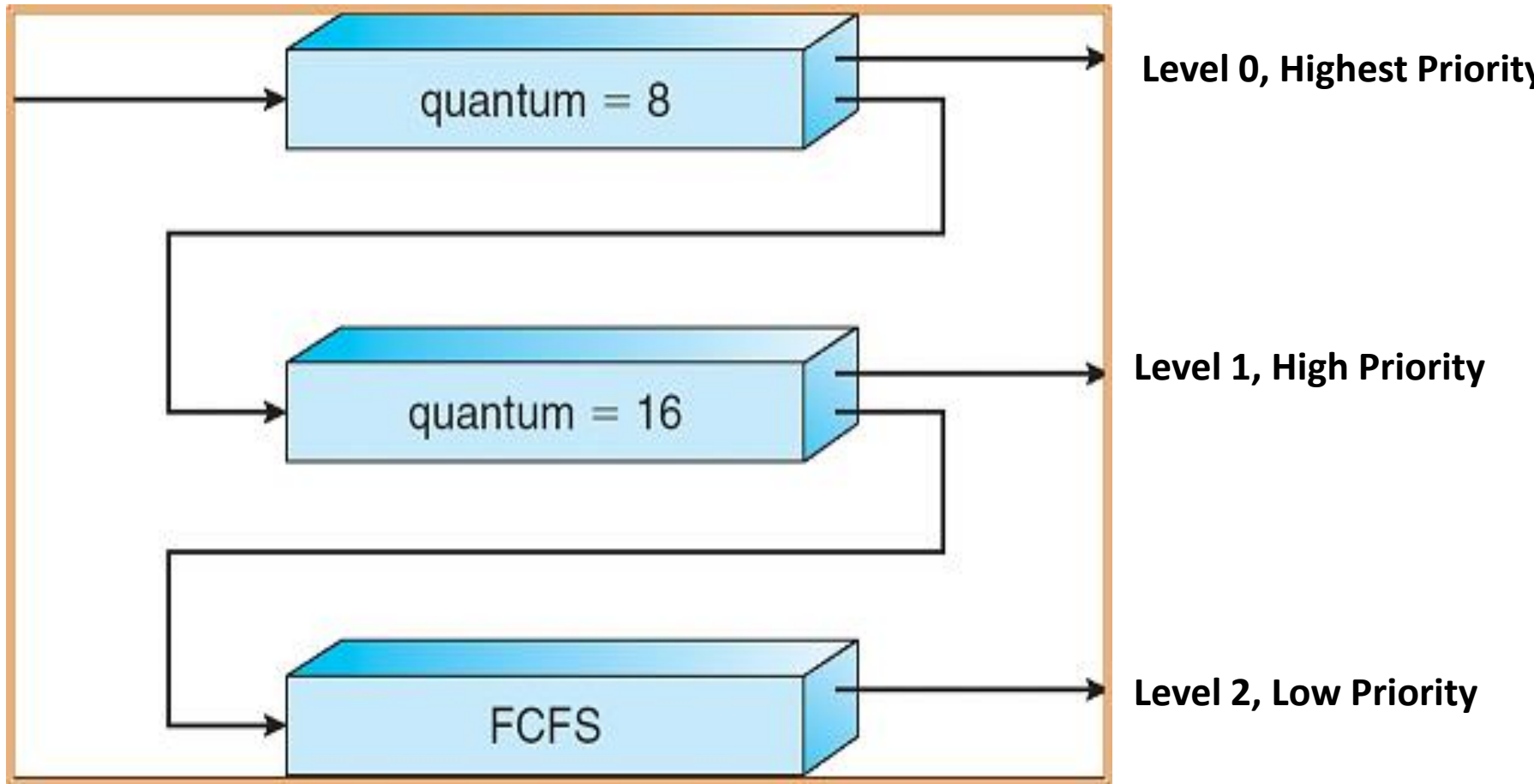
- ▶ There are *more than one* number of queues, each assigned a *different priority level*.
- ▶ Processes are executed from *top priority* queue to *low priority queue*.

- Rule 1: If $\text{Priority}(A) > \text{Priority}(B)$, A runs (B doesn't).
- Rule 2: If $\text{Priority}(A) = \text{Priority}(B)$, A & B run in RR.

MLFQ: Basic Rules

- ▶ MLFQ *varies the priority* (dynamic) of a job based on its observed behavior.
- ▶ If a process *repeatedly relinquishes the CPU*, MLFQ will keep *its priority high*.
- ▶ If a job uses the *CPU intensively* for long periods of time, MLFQ will *reduce its priority*.

MLFQ Example



- Note: 1. If a process *can't finish in 8 ms* then it would be *moved to Level 1* and So on.
2. Lower queues will run only after *upper queues are empty*.
3. If a process *repeatedly relinquishes the CPU*, MLFQ will keep *its priority high*.

Question?

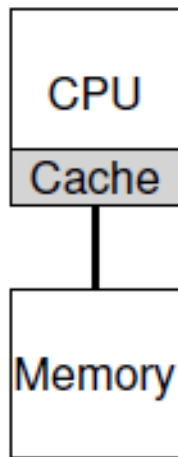
► Which CPU scheduling algorithm your system OS uses?

Note: It is the most *complex algorithm* too.

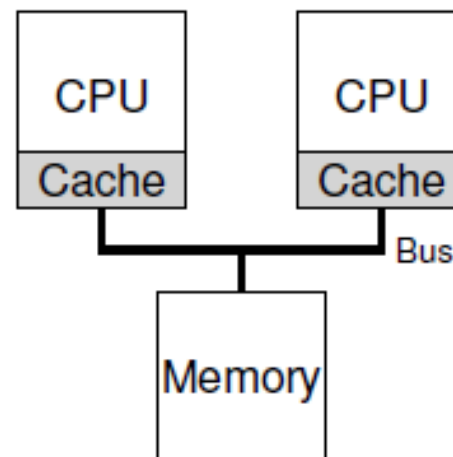
Problems With Current MLFQ

- ▶ It is the *most general CPU* scheduling algorithm.
- ▶ Also the most *complex algorithm*, since defining the best scheduler requires some means by which to select values for all parameters.
- ▶ *Starvation* to long jobs.
- ▶ A smart user could rewrite their program to **game the scheduler**.
- ▶ A process may change its behavior over time - from *CPU bound* to a phase of *interactivity*.

Uniprocessor vs Multiprocessor



Single CPU with Cache



Two CPUs with Caches

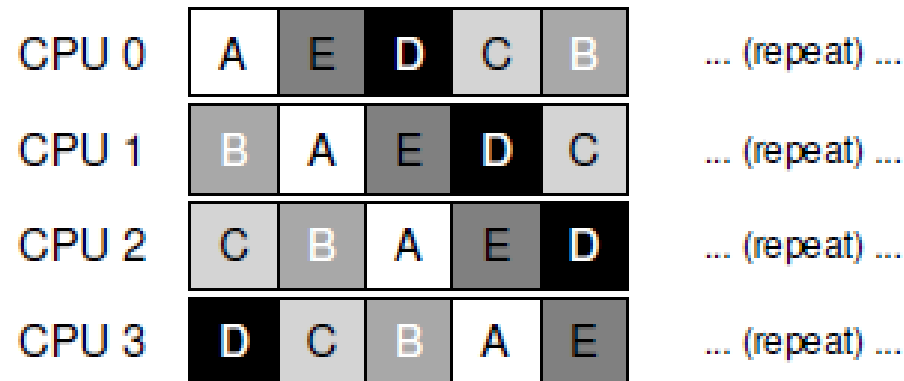
Multiprocessor Scheduling

- ▶ Single Queue Multiprocessor Scheduling
- ▶ Multi Queue Multiprocessor Scheduling

SQMS

- ▶ LTS loads all jobs to the common ready queue.
- ▶ However, there are multiple STSs.
 - + Simple to implement
 - Synchronization overhead (lock overhead)
 - Doesn't preserve Cache affinity

SQMS



RR Scheduling on 04 CPUs:

processes bounce from CPU to CPU.

Solution – SQMS must adopt some affinity mechanism (requires migration)

SQMS

CPU 0	A	E	A	A	A	... (repeat) ...
CPU 1	B	B	E	B	B	... (repeat) ...
CPU 2	C	C	C	E	C	... (repeat) ...
CPU 3	D	D	D	D	E	... (repeat) ...

Schedule with migration

MQMS

► 2 Processors: CPU0, CPU1.

► A goes to CPU0

► B goes to CPU1

► C goes to CPU0

► D goes to CPU1



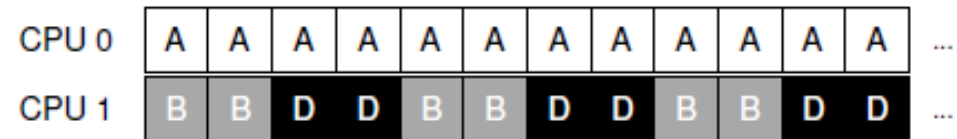
MQMS

- + Synchronization overhead (lock overhead)
- + Preserves Cache affinity
- Load imbalance

MQMS

- Load imbalance

- C finishes



- A finishes



MQMS

► Solution to load imbalance -> **migration**

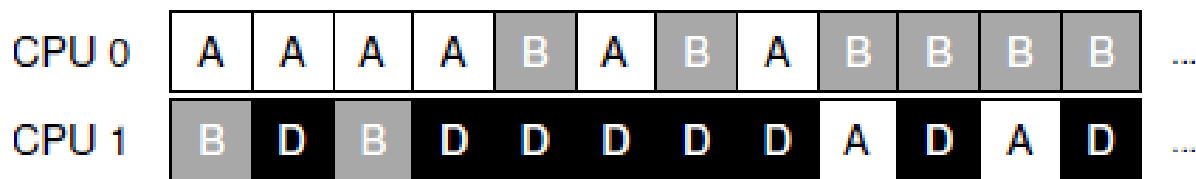


Solution: Simply move one of B or D to CPU 0

MQMS



- ▶ A single migration doesn't help.
- ▶ Solution: In every 50ms migrate one job.



Thank You